

# Contents

|                                                          |           |
|----------------------------------------------------------|-----------|
| <b>Les Fonctions</b>                                     | <b>3</b>  |
| <b>1 Pourquoi les fonctions ?</b>                        | <b>4</b>  |
| <b>2 Structure fondamentale d’une fonction</b>           | <b>4</b>  |
| Exemple minimal . . . . .                                | 4         |
| <b>3 Fonction avec valeur de retour</b>                  | <b>4</b>  |
| <b>4 Paramètres et passage d’arguments</b>               | <b>5</b>  |
| 4.1 Passage par objet . . . . .                          | 5         |
| Cas objet immuable . . . . .                             | 5         |
| Cas objet mutable . . . . .                              | 6         |
| 4.2 Paramètres par défaut . . . . .                      | 6         |
| Piège classique : valeur mutable par défaut . . . . .    | 6         |
| 4.3 Arguments positionnels et nommés . . . . .           | 7         |
| <b>5 Nombre variable d’arguments — *args et **kwargs</b> | <b>7</b>  |
| 5.1 *args — arguments positionnels variables . . . . .   | 7         |
| 5.2 **kwargs — arguments nommés variables . . . . .      | 8         |
| 5.3 Ordre obligatoire des paramètres . . . . .           | 8         |
| 5.4 Cas métier — fonction de log flexible . . . . .      | 9         |
| 5.5 Déballage à l’appel — * et ** . . . . .              | 9         |
| <b>6 Paramètres keyword-only</b>                         | <b>9</b>  |
| <b>7 Portée des variables — Scope</b>                    | <b>10</b> |
| global . . . . .                                         | 10        |
| nonlocal . . . . .                                       | 11        |
| <b>8 Fonctions imbriquées et closures</b>                | <b>11</b> |
| <b>9 Fonctions anonymes — lambda</b>                     | <b>12</b> |
| Annotations de type                                      | 12        |
| <b>1 1 Docstrings — format professionnel</b>             | <b>13</b> |
| <b>1 2 Récursivité</b>                                   | <b>13</b> |
| Limite de récursion . . . . .                            | 14        |
| <b>1 3 Générateurs et yield</b>                          | <b>14</b> |
| <b>1 4 Bonnes pratiques</b>                              | <b>14</b> |
| <b>1 5 Mini-exercices de production</b>                  | <b>15</b> |

|                                                            |           |
|------------------------------------------------------------|-----------|
| Exercice 1 — Prédiction . . . . .                          | 15        |
| Exercice 2 — Production . . . . .                          | 15        |
| Exercice 3 — Débogage . . . . .                            | 16        |
| <b>Synthèse</b>                                            | <b>17</b> |
| <b>20 Questions Intelligentes — Fonctions</b>              | <b>18</b> |
| <b>1) Valeur par défaut mutable — pourquoi dangereux ?</b> | <b>18</b> |
| Énoncé . . . . .                                           | 18        |
| Solution . . . . .                                         | 18        |
| <b>2) return vs print — différence fondamentale</b>        | <b>19</b> |
| Énoncé . . . . .                                           | 19        |
| Solution . . . . .                                         | 19        |
| <b>3) Fonction sans return explicite</b>                   | <b>19</b> |
| Énoncé . . . . .                                           | 19        |
| Solution . . . . .                                         | 19        |
| <b>4) Mutation d'un argument mutable</b>                   | <b>20</b> |
| Énoncé . . . . .                                           | 20        |
| Solution . . . . .                                         | 20        |
| <b>5) Ordre des paramètres avec défaut</b>                 | <b>20</b> |
| Énoncé . . . . .                                           | 20        |
| Solution . . . . .                                         | 20        |
| <b>6) *args — que contient réellement ?</b>                | <b>21</b> |
| Énoncé . . . . .                                           | 21        |
| Solution . . . . .                                         | 21        |
| <b>7) **kwargs — structure réelle</b>                      | <b>21</b> |
| Énoncé . . . . .                                           | 21        |
| Solution . . . . .                                         | 22        |
| <b>8) Combiner *args et **kwargs</b>                       | <b>22</b> |
| Énoncé . . . . .                                           | 22        |
| Solution . . . . .                                         | 22        |
| <b>9) Closure — piège avec boucle</b>                      | <b>23</b> |
| Énoncé . . . . .                                           | 23        |
| Solution . . . . .                                         | 23        |
| <b>10) Global mal utilisé</b>                              | <b>23</b> |
| Énoncé . . . . .                                           | 23        |
| Solution . . . . .                                         | 24        |

|                                                   |           |
|---------------------------------------------------|-----------|
| <b>11) nonlocal — usage réel</b>                  | <b>24</b> |
| Énoncé . . . . .                                  | 24        |
| Solution . . . . .                                | 24        |
| <b>12) Fonction comme objet</b>                   | <b>25</b> |
| Énoncé . . . . .                                  | 25        |
| Solution . . . . .                                | 25        |
| <b>13) Fonction passée en argument</b>            | <b>25</b> |
| Énoncé . . . . .                                  | 25        |
| Solution . . . . .                                | 25        |
| <b>14) yield vs return</b>                        | <b>26</b> |
| Énoncé . . . . .                                  | 26        |
| Solution . . . . .                                | 26        |
| <b>15) Générateur épuisé</b>                      | <b>27</b> |
| Énoncé . . . . .                                  | 27        |
| Solution . . . . .                                | 27        |
| <b>16) Lambda limitée</b>                         | <b>27</b> |
| Énoncé . . . . .                                  | 27        |
| Solution . . . . .                                | 27        |
| <b>17) Fonction récursive — condition d'arrêt</b> | <b>28</b> |
| Énoncé . . . . .                                  | 28        |
| Solution . . . . .                                | 28        |
| <b>18) Paramètres keyword-only</b>                | <b>28</b> |
| Énoncé . . . . .                                  | 28        |
| Solution . . . . .                                | 28        |
| <b>19) Shadowing d'une variable globale</b>       | <b>29</b> |
| Énoncé . . . . .                                  | 29        |
| Solution . . . . .                                | 29        |
| <b>20) functools.partial — spécialisation</b>     | <b>29</b> |
| Énoncé . . . . .                                  | 29        |
| Solution . . . . .                                | 30        |

## Les Fonctions

Le support est propriété de **Bechir Bejaoui**

---

# 1 Pourquoi les fonctions ?

Une fonction est une unité logique de traitement permettant :

- Structurer un programme
- Réutiliser du code
- Isoler la logique métier
- Tester indépendamment des blocs fonctionnels
- Réduire la complexité globale

Dans un contexte réel (ex : système RH, gestion de commandes, analyse de données), chaque action métier doit être encapsulée dans une fonction dédiée.

---

## 2 Structure fondamentale d'une fonction

```
def nom_fonction(parametres):  
    """  
    Description claire du rôle de la fonction.  
    """  
    # Corps de la fonction  
    return valeur    # optionnel
```

---

### Exemple minimal

```
"""  
Fonction sans paramètre ni retour.  
Une fonction sans return retourne implicitement None.  
"""
```

```
def dire_bonjour():  
    print("Bonjour")
```

```
dire_bonjour()
```

---

## 3 Fonction avec valeur de retour

```
"""  
return renvoie une valeur exploitable par l'appelant.  
print affiche uniquement - la valeur ne peut pas être réutilisée.  
"""
```

```

"""

def carre(x):
    return x * x

resultat = carre(5)
print(resultat)

# Différence return vs print
def addition_print(a, b):
    print(a + b) # affiche mais ne retourne rien

def addition_return(a, b):
    return a + b # valeur réutilisable

r = addition_return(3, 4)
print("Résultat réutilisable :", r * 2)

```

---

## 4 Paramètres et passage d'arguments

### 4.1 Passage par objet

Python transmet une référence vers l'objet.

#### Cas objet immuable

```

"""
Les entiers sont immuables.
Toute modification crée un nouvel objet local.
La variable externe n'est pas affectée.
"""

def incrementer(n):
    n += 1
    print("Dans fonction :", n)

a = 10
incrementer(a)
print("Après appel :", a) # inchangé

```

---

### Cas objet mutable

```
"""
Les listes sont mutables.
La modification impacte l'objet d'origine.
"""

def ajouter_element(liste):
    liste.append(100)

ma_liste = [1, 2, 3]
ajouter_element(ma_liste)
print(ma_liste)      # [1, 2, 3, 100]
```

---

## 4.2 Paramètres par défaut

```
"""
Les paramètres avec valeur par défaut sont optionnels à l'appel.
Ils doivent toujours être placés après les paramètres obligatoires.
"""

def saluer(nom, message="Bonjour"):
    print(f"{message}, {nom}")

saluer("Alice")          # Bonjour, Alice
saluer("Bob", "Bonsoir") # Bonsoir, Bob
```

---

### Piège classique : valeur mutable par défaut

```
"""
Les valeurs par défaut sont évaluées une seule fois à la définition.
Une liste par défaut est partagée entre tous les appels.
"""

def ajouter(element, liste=[]):
    liste.append(element)
    return liste

print(ajouter(1))      # [1]
print(ajouter(2))      # [1, 2] ← effet inattendu
```

Solution correcte :

```
"""
Utiliser None et créer une nouvelle liste dynamiquement.
```

```

"""

def ajouter(element, liste=None):
    if liste is None:
        liste = []
    liste.append(element)
    return liste

print(ajouter(1))    # [1]
print(ajouter(2))    # [2]

```

---

### 4.3 Arguments positionnels et nommés

```

"""
Positionnels : passés dans l'ordre de définition.
Nommés (keyword) : passés avec le nom du paramètre.
Les nommés peuvent être dans n'importe quel ordre.
"""

def decrire(nom, age, ville):
    print(f"{nom} - {age} - {ville}")

decrire("Alice", 25, "Paris")           # positionnel
decrire(age=30, nom="Bob", ville="Lyon") # nommés
decrire("Carol", ville="Marseille", age=22) # mixte

```

---

## 5 Nombre variable d'arguments — \*args et \*\*kwargs

### 5.1 \*args — arguments positionnels variables

```

"""
*args capture tous les arguments positionnels supplémentaires.
Le résultat est un TUPLE - pas une liste.
Le nom args est une convention - le * est obligatoire.
"""

def somme(*args):
    print("Type de args :", type(args))    # <class 'tuple'>
    print("Contenu      :", args)
    return sum(args)

```

```

print(somme(1, 2, 3))           # 6
print(somme(10, 20, 30, 40))   # 100
print(somme())                  # 0 - zéro argument accepté

```

---

## 5.2 \*\*kwargs — arguments nommés variables

```

"""
**kwargs capture tous les arguments nommés supplémentaires.
Le résultat est un DICT - clés = noms des arguments.
Le nom kwargs est une convention - le ** est obligatoire.
"""

def afficher_infos(**kwargs):
    print("Type de kwargs :", type(kwargs))    # <class 'dict'>
    print("Contenu      :", kwargs)
    for cle, valeur in kwargs.items():
        print(f" {cle} : {valeur}")

afficher_infos(nom="Alice", age=30, ville="Paris")
afficher_infos(produit="Laptop", prix=999.99)

```

---

## 5.3 Ordre obligatoire des paramètres

```

"""
L'ordre des paramètres est strict en Python :
1. Paramètres positionnels obligatoires
2. Paramètres avec valeur par défaut
3. *args
4. Paramètres keyword-only (après *)
5. **kwargs
"""

def fonction_complete(a, b=10, *args, option=False, **kwargs):
    print(f"a      = {a}")
    print(f"b      = {b}")
    print(f"args    = {args}")
    print(f"option  = {option}")
    print(f"kwargs   = {kwargs}")

fonction_complete(1, 2, 3, 4, option=True, x=99, y=100)

Sortie :

a      = 1

```



```
b      = 2
args   = (3, 4)
option = True
kwargs = {'x': 99, 'y': 100}
```

---

## 5.4 Cas métier — fonction de log flexible

```
"""
Cas réel : une fonction de log qui accepte
un message obligatoire et des métadonnées optionnelles.
"""

def log(message, niveau="INFO", **contexte):
    print(f"[{niveau}] {message}")
    for cle, valeur in contexte.items():
        print(f"  {cle} : {valeur}")

log("Démarrage", niveau="INFO")
log("Erreur critique", niveau="ERROR", module="auth", user_id=42)
log("Export terminé", fichier="rapport.csv", lignes=1500)
```

---

## 5.5 Déballage à l'appel — \* et \*\*

```
"""
* déballe une liste/tuple en arguments positionnels.
** déballe un dictionnaire en arguments nommés.
"""

def addition(a, b, c):
    return a + b + c

valeurs = [1, 2, 3]
print(addition(*valeurs))    # équivalent à addition(1, 2, 3)

params = {"a": 10, "b": 20, "c": 30}
print(addition(**params))    # équivalent à addition(a=10, b=20, c=30)
```

---

## 6 Paramètres keyword-only

```
"""
Tout paramètre après * doit être passé par nom.
"""
```

```

Améliore la lisibilité et évite les erreurs de position.
"""

def configurer(adresse, *, port=80, ssl=False):
    print(f"{adresse} - port={port} - ssl={ssl}")

configurer("localhost", port=8080, ssl=True)

# Incorrect - provoque TypeError
# configurer("localhost", 8080, True)

```

---

## 7 Portée des variables — Scope

```

"""
LEGB : Local → Enclosing → Global → Built-in
Python cherche une variable dans cet ordre.
"""

x = "global"

def externe():
    x = "enclosing"

    def interne():
        x = "local"
        print("Interne    :", x)

    interne()
    print("Externe    :", x)

externe()
print("Global    :", x)

```

---

**global**

```

"""
global permet de modifier une variable du scope global.
À éviter - préférer le retour de valeur.
"""

compteur = 0

```

```

def incrementer():
    global compteur
    compteur += 1

incrementer()
print(compteur)

# Meilleure pratique
def incrementer_propre(valeur):
    return valeur + 1

compteur = incrementer_propre(compteur)

```

---

## nonlocal

```

"""
nonlocal permet de modifier une variable
de la fonction englobante (enclosing scope).
"""

def externe():
    x = 5

    def interne():
        nonlocal x
        x += 2

    interne()
    print(x)    # 7

externe()

```

---

## 8 Fonctions imbriquées et closures

```

"""
Une closure capture les variables de son environnement.
La fonction interne garde accès aux variables de externe
même après que externe ait terminé.
"""

def multiplicateur(n):
    def interne(x):

```

```

        return x * n    # n est capturé depuis externe
    return interne

fois3 = multiplicateur(3)
fois5 = multiplicateur(5)

print(fois3(10))    # 30
print(fois5(10))    # 50

```

---

## 9 Fonctions anonymes — lambda

```

"""
lambda accepte uniquement une expression.
Utile pour les fonctions courtes passées en argument.
"""

carre = lambda x: x**2
print(carre(4))

# Usage courant avec sorted
personnes = [{"nom": "Bob", "age": 30}, {"nom": "Alice", "age": 25}]
triees = sorted(personnes, key=lambda p: p["age"])
print(triees)

```

---

## Annotations de type

```

"""
Les annotations améliorent la lisibilité et l'outillage statique.
Elles ne sont pas vérifiées à l'exécution.
"""

def addition(a: int, b: int) -> int:
    return a + b

def saluer(nom: str, fois: int = 1) -> None:
    for _ in range(fois):
        print(f"Bonjour, {nom}")

def somme_args(*args: float) -> float:
    return sum(args)

```

---

## 1 1 Docstrings — format professionnel

```
"""
Format Google - standard le plus répandu en entreprise.
"""

def calculer_remise(prix: float, taux: float) -> float:
    """
    Calcule le prix après application d'une remise.

    Args:
        prix : Prix initial hors remise.
        taux : Taux de remise entre 0 et 1 (ex: 0.10 pour 10%).

    Returns:
        Prix après remise.

    Raises:
        ValueError: Si le taux est hors de l'intervalle [0, 1].

    Example:
        >>> calculer_remise(100, 0.10)
        90.0
    """
    if not 0 <= taux <= 1:
        raise ValueError(f"Taux invalide : {taux}")
    return prix * (1 - taux)

print(calculer_remise(100, 0.10))
```

---

## 1 2 Récursivité

```
"""
Une fonction récursive s'appelle elle-même.
Deux éléments obligatoires :
- Condition d'arrêt (cas de base)
- Appel récursif qui converge vers le cas de base
Sans condition d'arrêt → RecursionError.
"""

def factoriel(n: int) -> int:
    """
    Calcule n! récursivement.
    factoriel(5) = 5 * 4 * 3 * 2 * 1 = 120
    """
```

```

"""
if n == 0:
    return 1          # cas de base
return n * factoriel(n - 1)  # appel récursif

print(factoriel(5))    # 120
print(factoriel(0))    # 1

```

### Limite de récursion

```

"""
Python limite la profondeur de récursion à 1000 par défaut.
"""

import sys
print(sys.getrecursionlimit())    # 1000

```

---

## 1 3 Générateurs et yield

```

"""
yield produit une valeur sans terminer la fonction.
Le générateur reprend à la ligne suivante au prochain appel.
Économie mémoire : les valeurs sont produites à la demande.
"""

def compteur(maximum: int):
    n = 0
    while n < maximum:
        yield n
        n += 1

for valeur in compteur(5):
    print(valeur)

# Un générateur ne peut être parcouru qu'une fois
g = (x for x in range(3))
print(list(g))    # [0, 1, 2]
print(list(g))    # [] ← épuisé

```

---

## 1 4 Bonnes pratiques

- Une fonction doit faire une seule chose

- Nom explicite et descriptif
  - Pas d'effet de bord non contrôlé
  - Éviter les variables globales — préférer le retour de valeur
  - Documenter avec docstring format Google
  - Annoter les types
  - Tester isolément
- 

## 1 5 Mini-exercices de production

### Exercice 1 — Prédiction

Prédire la sortie avant exécution :

```
def f(a, b=2, *args, opt=False, **kwargs):
    print(a, b, args, opt, kwargs)

f(1, 3, 4, 5, opt=True, x=9)

def ajouter(v, liste=[]):
    liste.append(v)
    return liste

print(ajouter(1))
print(ajouter(2))

def g():
    pass

print(g())
```

---

### Exercice 2 — Production

Écrire une fonction `rapport` qui :

1. Accepte un titre obligatoire
2. Accepte un niveau optionnel par nom uniquement ("INFO", "WARNING", "ERROR")
3. Accepte un nombre variable de messages positionnels
4. Accepte des métadonnées nommées variables
5. Affiche un rapport formaté

*# Votre solution ici*

Solution

```
"""
Fonction de rapport flexible utilisant *args et **kwargs.
"""

def rapport(titre: str, *messages: str, niveau: str = "INFO", **meta) -> None:
    """
    Génère un rapport formaté.

    Args:
        titre      : Titre du rapport.
        *messages  : Messages à inclure.
        niveau     : Niveau de rapport (keyword-only).
        **meta     : Métadonnées additionnelles.
    """
    print(f"\n[{niveau}] {titre}")
    print("-" * 30)
    for msg in messages:
        print(f"  • {msg}")
    if meta:
        print("  Métadonnées :")
        for cle, valeur in meta.items():
            print(f"    {cle} : {valeur}")

rapport("Démarrage", "Module chargé", "Config OK", niveau="INFO")
rapport(
    "Erreur critique",
    "Connexion échouée",
    "Retry impossible",
    niveau="ERROR",
    module="auth",
    user_id=42
)
```

---

### Exercice 3 — Débogage

Ce code contient 4 erreurs. Les identifier et les corriger.

```
def f(a=1, b):
    return a + b

def log(message, **kwargs, niveau="INFO"):
    print(message, kwargs, niveau)
```



```
def factoriel(n):
    return n * factoriel(n - 1)

def ajouter(v, liste=[]):
    liste.append(v)
    return liste

print(ajouter(1))
print(ajouter(2))
```

Solution

```
# Erreur 1 : paramètre sans défaut après paramètre avec défaut
def f(b, a=1):
    return a + b

# Erreur 2 : **kwargs doit être en dernier - keyword-only avant **kwargs
def log(message, *, niveau="INFO", **kwargs):
    print(message, kwargs, niveau)

# Erreur 3 : pas de condition d'arrêt → RecursionError
def factoriel(n):
    if n == 0:
        return 1
    return n * factoriel(n - 1)

# Erreur 4 : valeur mutable par défaut → partagée entre appels
def ajouter(v, liste=None):
    if liste is None:
        liste = []
    liste.append(v)
    return liste

print(ajouter(1))    # [1]
print(ajouter(2))    # [2]
```

---

## Synthèse

- Une fonction encapsule une logique réutilisable et testable
- **return** renvoie une valeur — sans **return** la fonction retourne **None**
- Python transmet une référence — les mutables sont modifiés en place
- **\*args** capture un **tuple** d'arguments positionnels variables

- **\*\*kwargs** capture un **dict** d'arguments nommés variables
- Ordre obligatoire : **pos**, **défaut**, **\*args**, **keyword-only**, **\*\*kwargs**
- Ne jamais utiliser une valeur mutable comme paramètre par défaut
- **global** est à éviter — préférer le retour de valeur
- **nonlocal** modifie une variable de la fonction englobante
- Les closures capturent les variables de leur environnement
- Les générateurs produisent les valeurs à la demande — économie mémoire
- Documenter avec le format Google et annoter les types

---

## 20 Questions Intelligentes — Fonctions

---

### 1) Valeur par défaut mutable — pourquoi dangereux ?

#### Énoncé

Pourquoi une liste en paramètre par défaut peut provoquer un bug cumulatif ?

#### Solution

```
"""
Explication :
Les valeurs par défaut sont évaluées une seule fois à la définition.
Une liste par défaut est partagée entre tous les appels.
"""

def ajouter(valeur, liste=[]):
    liste.append(valeur)
    return liste

print(ajouter(1))    # [1]
print(ajouter(2))    # [1, 2] ← effet inattendu

# Correction
def ajouter_propre(valeur, liste=None):
    if liste is None:
        liste = []
```

```

    liste.append(valeur)
    return liste

print(ajouter_propre(1))    # [1]
print(ajouter_propre(2))    # [2]

```

---

## 2) return vs print — différence fondamentale

### Énoncé

Pourquoi utiliser return plutôt que print dans une logique métier ?

### Solution

```

"""
Explication :
print affiche - la valeur ne peut pas être réutilisée.
return renvoie une valeur exploitable par l'appelant.
"""

def addition_print(a, b):
    print(a + b)          # affiche mais retourne None

def addition_return(a, b):
    return a + b          # valeur réutilisable

r = addition_return(5, 3)
print("Résultat :", r * 2)    # 16

```

---

## 3) Fonction sans return explicite

### Énoncé

Que retourne une fonction sans return ?

### Solution

```

"""
Explication :
Toute fonction retourne implicitement None
si aucun return n'est atteint.
"""

```

```
def test():
    x = 10

print(test())           # None
print(type(test()))    # <class 'NoneType'>
```

---

## 4) Mutation d'un argument mutable

### Énoncé

Pourquoi une liste est modifiée en dehors de la fonction ?

### Solution

```
"""
Explication :
Les objets mutables sont modifiés en place.
La fonction reçoit la référence vers le même objet.
"""

def ajouter_element(liste):
    liste.append(99)

data = [1, 2]
ajouter_element(data)
print(data)    # [1, 2, 99]
```

---

## 5) Ordre des paramètres avec défaut

### Énoncé

Pourquoi cette définition provoque une erreur ?

### Solution

```
"""
Explication :
Les paramètres sans défaut doivent précéder
ceux avec valeur par défaut.
"""
```

```

# Incorrect → SyntaxError
# def f(a=1, b):
#     pass

# Correct
def f(b, a=1):
    return a + b

print(f(5))          # 6
print(f(5, 10))      # 15

```

---

## 6) \*args — que contient réellement ?

### Énoncé

Quel est le type de \*args et comment l'utiliser ?

### Solution

```

"""
Explication :
*args capture les arguments positionnels supplémentaires.
Le résultat est un TUPLE - immuable.
On peut le parcourir, le déballer, le passer à sum().
"""

def demo(*args):
    print("Type      :", type(args))          # tuple
    print("Contenu  :", args)
    print("Somme    :", sum(args))

demo(1, 2, 3)
demo(10, 20, 30, 40, 50)
demo()      # tuple vide - aucune erreur

```

---

## 7) \*\*kwargs — structure réelle

### Énoncé

Quel est le type de \*\*kwargs et comment l'utiliser ?

## Solution

```
"""
Explication :
**kwargs capture les arguments nommés supplémentaires.
Le résultat est un DICT - clés = noms des arguments.
"""

def afficher(**kwargs):
    print("Type      :", type(kwargs))    # dict
    print("Contenu :", kwargs)
    for cle, valeur in kwargs.items():
        print(f" {cle} = {valeur}")

afficher(nom="Alice", age=30, ville="Paris")
afficher()    # dict vide - aucune erreur
```

---

## 8) Combiner \*args et \*\*kwargs

### Énoncé

Comment utiliser les deux ensemble ?

### Solution

```
"""
Explication :
*args et **kwargs peuvent coexister.
*args doit toujours précéder **kwargs.
"""

def rapport(*messages, **meta):
    print("Messages :")
    for m in messages:
        print(" -", m)
    print("Métadonnées :")
    for cle, val in meta.items():
        print(f" {cle} : {val}")

rapport(
    "Démarrage",
    "Config OK",
    module="auth",
    version="1.0"
```

)

---

## 9) Closure — piège avec boucle

### Énoncé

Pourquoi toutes les fonctions retournent la même valeur ?

### Solution

```
"""
Explication :
La variable i est capturée par référence.
Elle vaut 2 à la fin de la boucle pour toutes les lambdas.
"""

fonctions = []

for i in range(3):
    fonctions.append(lambda: i)

for f in fonctions:
    print(f())    # 2 2 2

# Correction : fixer la valeur via paramètre par défaut
fonctions = []

for i in range(3):
    fonctions.append(lambda i=i: i)

for f in fonctions:
    print(f())    # 0 1 2
```

---

## 10) Global mal utilisé

### Énoncé

Pourquoi éviter global ?

## Solution

```
"""
Explication :
Modifier une variable globale rend le code fragile.
Les effets de bord sont difficiles à tracer.
Préférer passer la valeur en paramètre et la retourner.
"""

# À éviter
compteur = 0

def incrementer():
    global compteur
    compteur += 1

# Meilleure pratique
def incrementer_propre(valeur: int) -> int:
    return valeur + 1

compteur = 0
compteur = incrementer_propre(compteur)
print(compteur)
```

---

## 11) nonlocal — usage réel

### Énoncé

Quand utiliser nonlocal ?

### Solution

```
"""
Explication :
nonlocal permet de modifier une variable
de la fonction englobante (enclosing scope).
Utile dans les closures qui maintiennent un état.
"""

def externe():
    x = 10

    def interne():
        nonlocal x
```



```

        x += 5

    interne()
    return x

print(externe())    # 15

```

---

## 12) Fonction comme objet

### Énoncé

Peut-on stocker une fonction dans une variable ?

### Solution

```

"""
Explication :
Les fonctions sont des objets de première classe.
Elles peuvent être stockées, passées en argument, retournées.
"""

def dire():
    print("Bonjour")

f = dire
f()

# Stockage dans une liste
actions = [lambda: print("A"), lambda: print("B")]
for action in actions:
    action()

```

---

## 13) Fonction passée en argument

### Énoncé

Comment créer une fonction générique ?

### Solution

```

"""
Explication :

```

*Passer une fonction en paramètre permet de généraliser le comportement.*

```
"""  
  
def appliquer(fonction, valeur):  
    return fonction(valeur)  
  
print(appliquer(lambda x: x * 2, 5))  
print(appliquer(lambda x: x ** 2, 4))  
print(appliquer(str.upper, "python"))
```

---

## 14) yield vs return

### Énoncé

Quelle différence comportementale ?

### Solution

```
"""  
Explication :  
return termine la fonction et renvoie une valeur.  
yield produit une valeur et suspend - reprend au prochain appel.  
Les générateurs économisent la mémoire.  
"""  
  
def avec_return(n):  
    return list(range(n))    # crée toute la liste en mémoire  
  
def avec_yield(n):  
    i = 0  
    while i < n:  
        yield i                # produit à la demande  
        i += 1  
  
for x in avec_yield(5):  
    print(x)
```

---

## 15) Générateur épuisé

### Énoncé

Pourquoi le second parcours ne donne rien ?

### Solution

```
"""
Explication :
Un générateur ne peut être parcouru qu'une seule fois.
Une fois épuisé, il ne produit plus rien.
Pour réitérer, recréer le générateur.
"""

g = (x for x in range(3))

print(list(g))    # [0, 1, 2]
print(list(g))    # [] ← épuisé

# Recréer si nécessaire
g = (x for x in range(3))
print(list(g))    # [0, 1, 2]
```

---

## 16) Lambda limitée

### Énoncé

Pourquoi lambda ne peut contenir plusieurs instructions ?

### Solution

```
"""
Explication :
lambda accepte uniquement une expression - pas des instructions.
Pour une logique multi-lignes, utiliser def.
"""

# Lambda simple
max2 = lambda a, b: a if a > b else b
print(max2(3, 8))

# Cas complexe → préférer def
def max_propre(a, b):
```

```

    if a > b:
        return a
    return b

print(max_propre(3, 8))

```

---

## 17) Fonction récursive — condition d'arrêt

### Énoncé

Que se passe-t-il sans condition d'arrêt ?

### Solution

```

"""
Explication :
Sans condition d'arrêt → RecursionError.
Python limite la profondeur à 1000 par défaut.
La condition d'arrêt est le cas de base.
"""

import sys
print("Limite récursion :", sys.getrecursionlimit())

def factoriel(n: int) -> int:
    if n == 0:          # cas de base - condition d'arrêt
        return 1
    return n * factoriel(n - 1)

print(factoriel(5))    # 120

```

---

## 18) Paramètres keyword-only

### Énoncé

Pourquoi forcer l'usage par nom ?

### Solution

```

"""
Explication :
Tout paramètre après * doit être passé par nom.

```

```

Améliore la lisibilité et évite les erreurs de position.
"""

def configurer(adresse: str, *, port: int = 80, ssl: bool = False) -> None:
    print(f"{adresse} - port={port} - ssl={ssl}")

configurer("localhost", port=8080, ssl=True)

try:
    configurer("localhost", 8080)    # TypeError
except TypeError as e:
    print("Erreur :", e)

```

---

## 19) Shadowing d'une variable globale

### Énoncé

Pourquoi cette variable globale n'est pas modifiée ?

### Solution

```

"""
Explication :
Une affectation dans une fonction crée une variable locale.
Elle masque (shadow) la variable globale sans la modifier.
Utiliser global pour modifier explicitement.
"""

x = 10

def f():
    x = 5          # variable locale - masque la globale
    print("Locale :", x)

f()
print("Globale :", x)    # 10 - inchangée

```

---

## 20) functools.partial — spécialisation

### Énoncé

Comment préconfigurer une fonction ?

## Solution

```
"""
Explication :
partial crée une nouvelle fonction avec certains arguments
déjà fixés.
Utile pour spécialiser des fonctions génériques.
"""

from functools import partial

def multiplier(a: int, b: int) -> int:
    return a * b

fois2 = partial(multiplier, 2)
fois10 = partial(multiplier, 10)

print(fois2(5))      # 10
print(fois10(5))     # 50

# Cas métier : formater des prix
def formater(valeur: float, symbole: str, decimales: int) -> str:
    return f"{valeur:.{decimales}f} {symbole}"

formater_euro = partial(formater, symbole="€", decimales=2)
print(formater_euro(19.9876))    # 19.99 €
```